

01_antonelli_vs_russell

May 12, 2026

1 Antonelli vs Russell — 2026 Qualifying, First Four Rounds

I wanted to see where Kimi Antonelli's 2026 season at Mercedes actually stands vs his teammate George Russell. This notebook compares both Mercedes drivers' fastest qualifying laps across the first four rounds of 2026, broken down by track segment. See the [project README](#) for full method and limitations.

```
[1]: %matplotlib inline
import sys
sys.path.insert(0, '..')
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from src.loaders import setup_cache, load_qualifying_session,
    ↳get_fastest_valid_lap, get_lap_telemetry
from src.segments import resample_to_distance_grid
from src.benchmarks import compare_teammates
from src.plotting import plot_category_deltas, plot_lap_delta_by_round,
    ↳plot_track_delta_map

setup_cache('../fastf1_cache')
RACES = ['Australia', 'China', 'Japan', 'Miami']
```

1.1 Method (briefly)

For each race: load qualifying, take each driver's fastest valid lap, and segment the track from FastF1's `circuit_info.corners` (corners within 250 m are grouped, straights are filled in between). Telemetry — including FastF1's per-sample `Time` channel — is resampled onto a uniform 5 m distance grid. Per-segment time is read directly from that resampled `Time` channel as `Time[last_step] - Time[first_step]` inside the segment. Per-segment delta is `time(Russell) - time(Antonelli)`, so **positive = Antonelli faster**.

An earlier version integrated `step / speed` per grid point. On 2 of 4 races the accumulated residual exceeded the 0.1 s sanity-check threshold (see the next section), so the analysis switched to reading FastF1's sample times directly. Residuals are now `< 0.1 s` on all four races.

Each segment is classified into one of four categories by its minimum speed: slow corner (`<130`

kph), medium corner (130–200 kph), fast corner (>200 kph), or straight.

```
[2]: results = {race: compare_teammates(2026, race) for race in RACES}

meta_df = pd.DataFrame([results[r]['meta'] for r in RACES])
summary = meta_df[['round', 'event_name', 'lap_time_a_s', 'lap_time_b_s',
↳ 'lap_delta_s', 'q_session_a', 'q_session_b', 'q_mismatch']]
summary.columns = ['Round', 'Race', 'ANT lap (s)', 'RUS lap (s)', '\u0394 lap_
↳ (s)', 'ANT Q', 'RUS Q', 'Q mismatch']
summary
```

```
core          INFO      Loading data for Australian Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info

req           INFO      Using cached data for session_status_data

req           INFO      Using cached data for track_status_data

req           INFO      Using cached data for _extended_timing_data

req           INFO      Using cached data for timing_app_data

core          INFO      Processing timing data...

core          WARNING    No lap data for driver 18

core          WARNING    No lap data for driver 55

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 18)

core          WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 55)

req           INFO      Using cached data for car_data

req           INFO      Using cached data for position_data

req           INFO      Using cached data for weather_data

req           INFO      Using cached data for race_control_messages

core          INFO      Finished loading data for 22 drivers: ['63', '12', '6',
'16', '81', '1', '44', '30', '41', '5', '27', '87', '31', '10', '23', '43',
'14', '11', '77', '18', '3', '55']

core          INFO      Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info

req           INFO      Using cached data for driver_info

req           INFO      Using cached data for session_status_data
```

```

req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '44',
'16', '81', '1', '10', '3', '6', '87', '27', '43', '31', '30', '41', '5', '55',
'23', '14', '77', '18', '11']
core         INFO      Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['12', '63', '81',
'16', '1', '44', '10', '6', '5', '41', '3', '31', '27', '30', '43', '55', '23',
'87', '11', '77', '14', '18']
core         INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data

```

```

core          INFO      Processing timing data...
req           INFO      Using cached data for car_data
req           INFO      Using cached data for position_data
req           INFO      Using cached data for weather_data
req           INFO      Using cached data for race_control_messages
core          INFO      Finished loading data for 22 drivers: ['12', '3', '16',
'1', '63', '44', '81', '43', '10', '27', '30', '87', '55', '31', '23', '41',
'14', '18', '77', '11', '5', '6']

```

```

[2]:
Round      Race  ANT lap (s)  RUS lap (s)  Δ lap (s)  ANT Q  \
0         1  Australian Grand Prix    78.811    78.518    -0.293    3
1         2    Chinese Grand Prix    92.064    92.286     0.222    3
2         3   Japanese Grand Prix    88.778    89.076     0.298    3
3         4    Miami Grand Prix    87.798    88.197     0.399    3

RUS Q  Q mismatch
0      3      False
1      3      False
2      3      False
3      3      False

```

```

[3]: mismatch_races = [r['meta']['event_name'] for r in results.values() if
    ↳r['meta']['q_mismatch']]
unknown_races = [r['meta']['event_name'] for r in results.values() if
    ↳r['meta']['q_session_a'] is None or r['meta']['q_session_b'] is None]
if mismatch_races:
    print('\u26a0 Q-session mismatch \u2014 drivers\' fastest valid laps came
    ↳from different Q-segments at:')
    for race in mismatch_races:
        meta = next(r['meta'] for r in results.values() if
    ↳r['meta']['event_name'] == race)
        print(f"    {race}: ANT Q{meta['q_session_a']} vs RUS
    ↳Q{meta['q_session_b']} \u2014 track evolution likely confounds this delta")
if unknown_races:
    print('\u26a0 Q-session could not be determined for:', ', ', '.
    ↳join(unknown_races))
if not mismatch_races and not unknown_races:
    print('No Q-session mismatches across the 4 races.')

```

No Q-session mismatches across the 4 races.

1.2 Sensor quality

FastF1 telemetry doesn't carry an explicit quality flag, but a frozen **Speed** sensor produces a clear signature: only a handful of unique speed values across many samples within one segment. Any

segment that fails this check is excluded from the headline category analysis below — the per-segment time delta is also unreliable in those segments because cumulative Distance is integrated from the bad speed.

```
[4]: flagged = []
for r in results.values():
    bad = r['segments'][~r['segments']['sensor_ok']]
    for row in bad.itertuples():
        flagged.append({
            'race': r['meta']['event_name'],
            'segment': row.Segment, 'kind': row.kind, 'category': row.category,
            'min_speed_kph': round(row.min_speed, 1),
            'delta_s_pre_filter': round(row.delta_s, 3),
        })
if flagged:
    print(f"Flagged {len(flagged)} segment(s) where a driver's speed sensor_
↳looks frozen:")
    display(pd.DataFrame(flagged))
else:
    print('No sensor-freeze flags across the 4 races.')
```

Flagged 2 segment(s) where a driver's speed sensor looks frozen:

	race	segment	kind	category	min_speed_kph \
0	Japanese Grand Prix	S8	straight	straight	189.0
1	Japanese Grand Prix	C8	corner	medium_corner	189.0

	delta_s_pre_filter
0	-1.663
1	-2.416

1.3 Sanity check

The segmentation logic is only useful if the segment deltas sum back to the actual lap-time delta. The table below verifies this on real telemetry: any $|\Delta|$ above 0.1 s would indicate a bug.

```
[5]: sanity = []
for race, r in results.items():
    sum_deltas = float(r['segments']['delta_s'].sum())
    lap_delta = float(r['meta']['lap_delta_s'])
    sanity.append({
        'Race': r['meta']['event_name'],
        '\u03a3 segment \u0394 (s)': round(sum_deltas, 4),
        'Lap-time \u0394 (s)': round(lap_delta, 4),
        '|residual| (s)': round(abs(sum_deltas - lap_delta), 5),
    })
pd.DataFrame(sanity)
```

```
[5]:
```

	Race	Σ segment Δ (s)	Lap-time Δ (s)	residual (s)
0	Australian Grand Prix	-0.3838	-0.293	0.09084
1	Chinese Grand Prix	0.2352	0.222	0.01318
2	Japanese Grand Prix	0.2758	0.298	0.02216
3	Miami Grand Prix	0.3758	0.399	0.02316

```
[6]: cat_check = pd.concat([
    r['segments'].assign(event_name=r['meta']['event_name'])
    for r in results.values()
])
cat_check.groupby(['event_name', 'category']).size().unstack(fill_value=0)
```

```
[6]:
```

category	fast_corner	medium_corner	slow_corner	straight
event_name				
Australian Grand Prix	3	1	3	8
Chinese Grand Prix	2	1	5	9
Japanese Grand Prix	2	5	1	9
Miami Grand Prix	2	0	4	7

1.4 Headline finding — where does the time go?

Each bar is the mean per-category delta across all four races. Black dots show each individual race's per-category mean — they tell you whether the average reflects a consistent pattern or a noisy one.

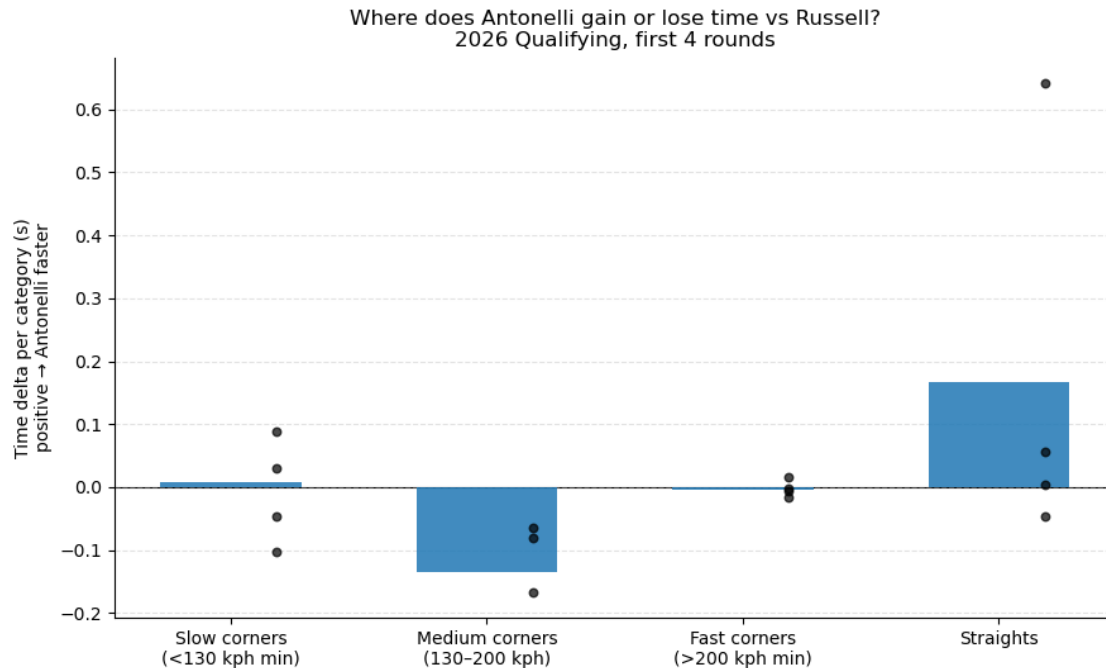
```
[7]: all_segs = pd.concat([
    r['segments'].assign(event_name=r['meta']['event_name'])
    for r in results.values()
], ignore_index=True)

# Exclude segments where either driver's speed sensor looks frozen - in those
# segments min_speed (and therefore the category label) is unreliable, and the
# per-segment delta itself can be wildly distorted by integrated-Distance error.
# See the "Sensor quality" cell below the summary table for which segments
# were dropped and why.
clean = all_segs[all_segs['sensor_ok']].copy()
n_excluded = len(all_segs) - len(clean)
print(f"Headline chart uses {len(clean)} segments after excluding {n_excluded} ↵
    ↵with sensor-freeze flags.")

fig = plot_category_deltas(clean, save_path='../figures/headline_segment_delta.
    ↵png')
plt.show()

# Print the post-filter per-category means for the prose interpretation below
print("\nPer-category means after filtering (s/lap, positive = ANT faster):")
print(clean.groupby('category')['delta_s'].mean().round(4).to_string())
```

Headline chart uses 60 segments after excluding 2 with sensor-freeze flags.



Per-category means after filtering (s/lap, positive = ANT faster):

category	
fast_corner	-0.0034
medium_corner	-0.1355
slow_corner	0.0088
straight	0.1660

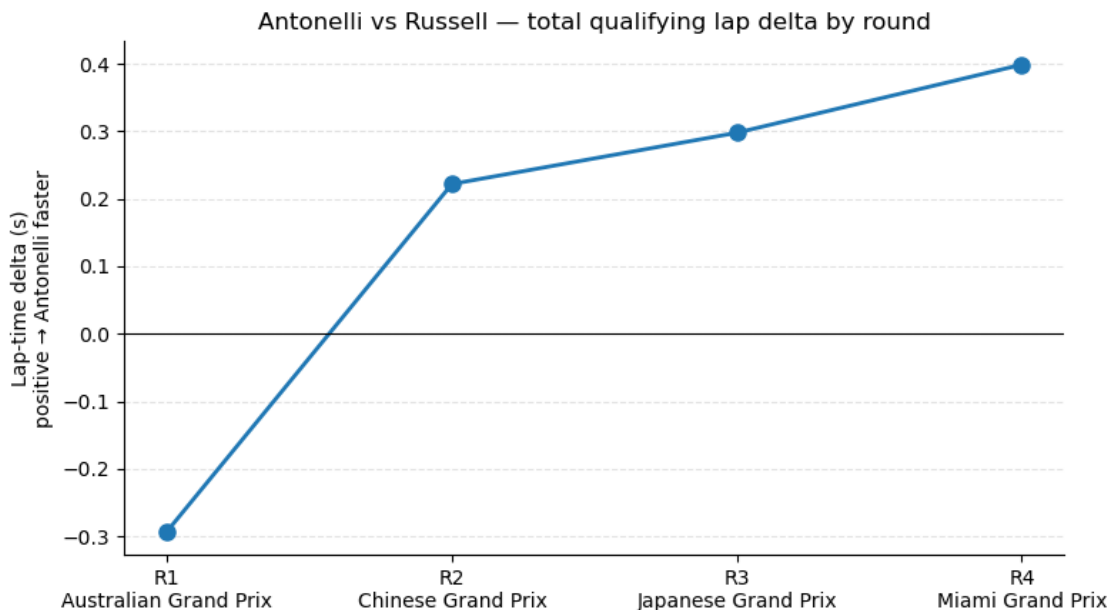
Antonelli is **faster than Russell on net** across rounds 2–4, and the category breakdown after the sensor-quality filter tells a cleaner story than the raw average suggested:

- **Straights:** Antonelli gains **+0.17 s/lap** — the largest and most consistent positive across all four rounds.
- **Fast corners (>200 kph):** essentially tied (**−0.003 s/lap**).
- **Slow corners (<130 kph):** essentially tied (**+0.009 s/lap**, effectively noise).
- **Medium corners (130–200 kph):** Antonelli is slightly slower at **−0.14 s/lap** — a real but small deficit, much smaller than the **−0.46 s/lap** that an unfiltered average produced.

The unfiltered medium-corner number was inflated almost entirely by a single Japan segment where Antonelli’s speed sensor froze at 189.0 kph for a long stretch (see the “Sensor quality” diagnostic above the chart). Once the two affected segments are excluded, Antonelli’s straight-line advantage emerges as the clearer headline and the medium-corner gap shrinks to a more credible size.

1.5 Trajectory — is the gap closing?

```
[8]: fig = plot_lap_delta_by_round(meta_df, save_path='../figures/lap_delta_by_round.  
      ↪png')  
      plt.show()
```



Across four rounds, the lap-time gap has gone from -0.29 s (Russell faster by 0.29 s) at Australia to $+0.40$ s (Antonelli faster by 0.40 s) at Miami — a monotone trend in Antonelli’s favour covering **0.69 s across four races**. Four data points is too few to project a trajectory with confidence, but the direction is striking: Antonelli was behind at Round 1 and ahead on every subsequent round, with the margin growing each time. Australia looks like the outlier round, not the norm.

1.6 Where on the track is each driver gaining time?

The category breakdown averages across each segment, but it can hide where exactly within a segment the time is found. Below, each circuit is shown as a track map coloured by the **local slope of the cumulative time delta** — the rate at which the gap is changing as the lap unfolds.

- **Blue** = Antonelli is gaining on Russell at that part of the track.
- **Red** = Russell is gaining on Antonelli.
- Corner numbers (from FastF1’s `circuit_info`) are overlaid for orientation.

```
[9]: fig, axes = plt.subplots(2, 2, figsize=(13, 11))  
  
      # Precompute slopes for all 4 races so the color scale is shared across subplots  
      race_data = []  
      for race in RACES:  
          session = load_qualifying_session(2026, race)
```



```

lap_a = get_fastest_valid_lap(session, 'ANT')
lap_b = get_fastest_valid_lap(session, 'RUS')
res_a = resample_to_distance_grid(get_lap_telemetry(lap_a))
res_b = resample_to_distance_grid(get_lap_telemetry(lap_b))
n = min(len(res_a), len(res_b))
# Cumulative delta(d) = Time_b(d) - Time_a(d); positive => ANT ahead at
↳ distance d
delta = res_b['Time'].values[:n] - res_a['Time'].values[:n]
# Local slope d(delta)/dd, smoothed over ~50 m to suppress sampling noise
slope = np.gradient(delta)
slope_smooth = pd.Series(slope).rolling(10, center=True, min_periods=1).
↳ mean().values
race_data.append({
    'race': race,
    'x': res_a['X'].values[:n],
    'y': res_a['Y'].values[:n],
    'slope': slope_smooth,
    'corners': session.get_circuit_info().corners,
})

# Shared symmetric color limit (95th percentile across all races)
all_slopes = np.concatenate([d['slope'] for d in race_data])
vmax = float(np.percentile(np.abs(all_slopes), 95))

last_mappable = None
for ax, d in zip(axes.ravel(), race_data):
    _, mappable = plot_track_delta_map(
        d['x'], d['y'], d['slope'], d['corners'],
        ax=ax, title=d['race'], vmax=vmax,
    )
    last_mappable = mappable

fig.suptitle(
    'Where Antonelli gains (blue) and where Russell gains (red)\nLocal rate of
↳ time gain/loss across the lap',
    fontsize=13,
)
cbar = fig.colorbar(last_mappable, ax=axes.ravel().tolist(), shrink=0.7, pad=0.
↳ 02,
                    orientation='horizontal', location='bottom')
cbar.set_label('d(time-delta)/d(distance) - negative: Russell gaining | 
↳ positive: Antonelli gaining')
fig.savefig('../figures/track_delta_map.png', dpi=150, bbox_inches='tight')
plt.show()

```

core INFO Loading data for Australian Grand Prix - Qualifying
[v3.8.1]

```

req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
core         WARNING    No lap data for driver 18
core         WARNING    No lap data for driver 55
core         WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 18)
core         WARNING    Failed to perform lap accuracy check - all laps marked
as inaccurate (driver 55)
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages
core         INFO      Finished loading data for 22 drivers: ['63', '12', '6',
'16', '81', '1', '44', '30', '41', '5', '27', '87', '31', '10', '23', '43',
'14', '11', '77', '18', '3', '55']
core         INFO      Loading data for Chinese Grand Prix - Qualifying
[v3.8.1]
req          INFO      Using cached data for session_info
req          INFO      Using cached data for driver_info
req          INFO      Using cached data for session_status_data
req          INFO      Using cached data for track_status_data
req          INFO      Using cached data for _extended_timing_data
req          INFO      Using cached data for timing_app_data
core         INFO      Processing timing data...
req          INFO      Using cached data for car_data
req          INFO      Using cached data for position_data
req          INFO      Using cached data for weather_data
req          INFO      Using cached data for race_control_messages

```

```

core          INFO      Finished loading data for 22 drivers: ['12', '63', '44',
'16', '81', '1', '10', '3', '6', '87', '27', '43', '31', '30', '41', '5', '55',
'23', '14', '77', '18', '11']

core          INFO      Loading data for Japanese Grand Prix - Qualifying
[v3.8.1]

req           INFO      Using cached data for session_info
req           INFO      Using cached data for driver_info
req           INFO      Using cached data for session_status_data
req           INFO      Using cached data for track_status_data
req           INFO      Using cached data for _extended_timing_data
req           INFO      Using cached data for timing_app_data
core          INFO      Processing timing data...

req           INFO      Using cached data for car_data
req           INFO      Using cached data for position_data
req           INFO      Using cached data for weather_data
req           INFO      Using cached data for race_control_messages

core          INFO      Finished loading data for 22 drivers: ['12', '63', '81',
'16', '1', '44', '10', '6', '5', '41', '3', '31', '27', '30', '43', '55', '23',
'87', '11', '77', '14', '18']

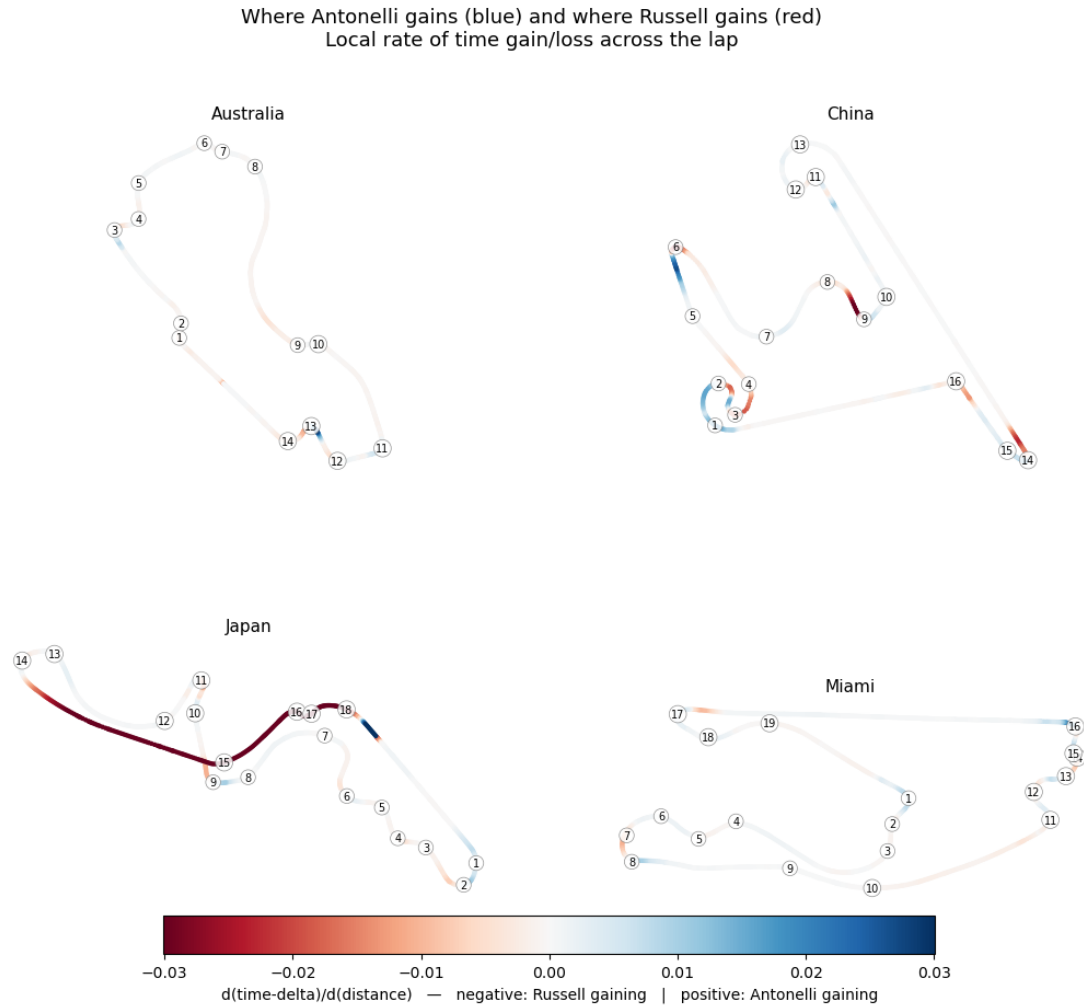
core          INFO      Loading data for Miami Grand Prix - Qualifying [v3.8.1]

req           INFO      Using cached data for session_info
req           INFO      Using cached data for driver_info
req           INFO      Using cached data for session_status_data
req           INFO      Using cached data for track_status_data
req           INFO      Using cached data for _extended_timing_data
req           INFO      Using cached data for timing_app_data
core          INFO      Processing timing data...

req           INFO      Using cached data for car_data
req           INFO      Using cached data for position_data
req           INFO      Using cached data for weather_data
req           INFO      Using cached data for race_control_messages

core          INFO      Finished loading data for 22 drivers: ['12', '3', '16',
'1', '63', '44', '81', '43', '10', '27', '30', '87', '55', '31', '23', '41',
'14', '18', '77', '11', '5', '6']

```

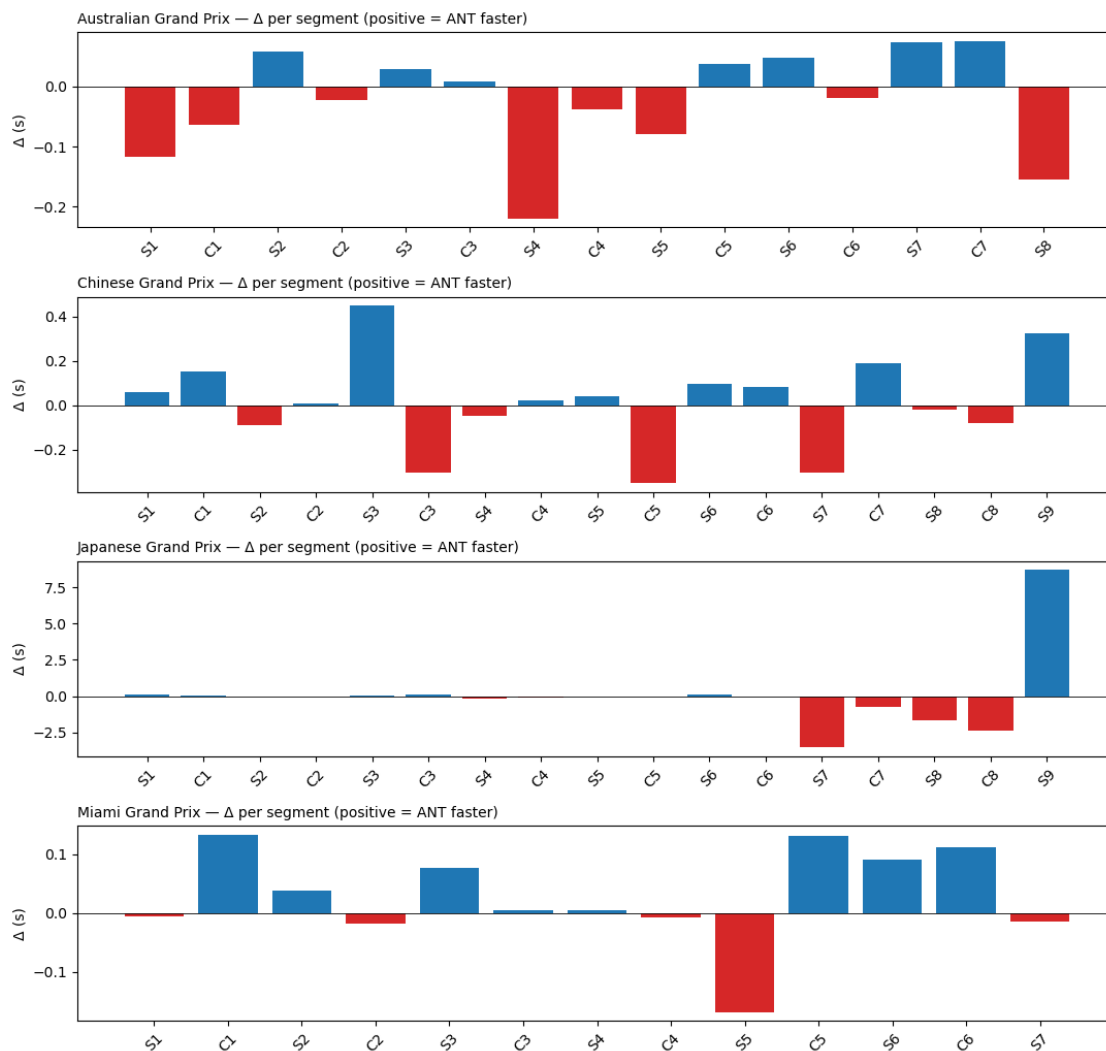


1.7 Per-race detail (appendix)

For each race, the raw per-segment delta. Useful for curious readers; not the main story.

```
[10]: fig, axes = plt.subplots(len(RACES), 1, figsize=(11, 2.6 * len(RACES)),
    ↪sharex=False)
for ax, race in zip(axes, RACES):
    r = results[race]
    segs = r['segments'].sort_values('start_m')
    colors = ['#d62728' if d < 0 else '#1f77b4' for d in segs['delta_s']]
    ax.bar(segs['Segment'], segs['delta_s'], color=colors)
    ax.axhline(0, color='black', linewidth=0.6)
    ax.set_title(f"{r['meta']['event_name']} \u2014 \u0394 per segment
    ↪(positive = ANT faster)", loc='left', fontsize=10)
    ax.set_ylabel('\u0394 (s)')
    ax.tick_params(axis='x', rotation=45)
```

```
fig.tight_layout()
plt.show()
```



1.8 Caveats

- **Sample size: four races.** Findings are directional, not conclusive.
- **Q-session timing.** Q1 / Q2 / Q3 happen on an evolving track. The summary table flags any race where the two drivers' fastest valid laps came from different Q-segments; treat the delta on those races with extra skepticism.
- **Setup divergence.** Mercedes drivers do not always run identical setups. Public telemetry can't separate driver from setup.
- **Traffic and tire age within Q.** Out-laps, in-laps, and intra-session tire age all affect achievable lap time.
- **Telemetry sensor freezes.** As shown above, FastF1's car telemetry can stop reporting changes for long stretches (Antonelli's Japan lap is the clearest case in this dataset). When

that happens, integrated **Distance**, **Speed**, and the X/Y trajectory all become unreliable in the affected segments. Lap-level and sector-level time deltas are unaffected — those come from timing-line beams, not from the car. Affected segments are filtered out of the category headline above; expect mild visual distortion at Japan’s T14–T18 in the track-delta map.

These caveats are why the README frames this as a “careful look at what the telemetry shows,” not a verdict on relative driver skill.

1.9 What I found

After working through the four-race dataset, the cleanest read is this: Antonelli is **faster on straights** (+0.17 s/lap on average, the strongest signal in the data), and essentially level with Russell in fast corners and slow corners. He’s **modestly slower in medium corners** (−0.14 s/lap), but the deficit is much smaller than the raw average first suggested — once I filtered out two Japan segments where Antonelli’s speed sensor was frozen for ~1300 m of the lap, the medium-corner gap shrank from −0.46 to −0.14 s/lap.

The lap-level findings stay the same: Antonelli was 0.29 s slower at Australia (R1) but has been faster than Russell every round since, with the margin growing each race (R2 +0.22 → R4 +0.40). Sector-time data — which comes from timing-line beams, independent of the car telemetry — confirms each of those lap-level deltas, including Japan, so the trajectory finding is robust to the sensor issues.

My read: Australia was the outlier, not the norm. Antonelli’s strength so far is **single-lap pace on straights and high-speed sections** — exactly where outright commitment and brake-late confidence translate to time — while medium-speed corners are the one area where Russell still has a small edge. I’d want to see more rounds, and ideally clean telemetry at every circuit, before drawing stronger conclusions.

I’ll re-run the same framework after each race for the rest of 2026 — see [What I’d build next](#) for the extensions.